Pocket Jungle

Minigames, phone sensors and validation guide

Game	Pocket Jungle: a collection of 3 phone minigames (plus 1 empty slot for a future game)
Minigames	Egg Run (Lina) · Sound Copy (Ons) · Photo Challenge (Eya)
Engine	Unity 6 (6000.6.3f1), Universal Render Pipeline (URP)
Platform	Android phone, held sideways (landscape)
Tested phone	Samsung Galaxy A17

Everything here is explained in simple words first, then with the exact technical details, so you can answer both easy and deep questions during the validation.

Contents

1. **The big picture**: what Pocket Jungle is and how the parts fit together
2. **Phone sensors explained simply**: what each sensor measures, and which game uses it
3. **Egg Run**: tilt, walk, shake the bird off, and block the sun with the camera
4. **Sound Copy**: imitate a sound; microphone, raise-to-record, spin-the-phone and the AI judge
5. **Photo Challenge**: find an object and photograph it; camera and image AI
6. **Shared systems**: menus, settings, saving, pause, sound and music
7. **Testing tools and building the app**
8. **Validation questions and answers**: everything that can be asked
9. **Glossary**: every technical word in one sentence

> **The whole project in 4 sentences**
>
> Pocket Jungle is a mobile game made of small games (minigames). Each minigame uses the phone's sensors as the controller instead of only tapping the screen: tilting, shaking, covering the camera, raising the phone like a microphone, spinning it on a table and taking photos. A shared menu system (main menu, level list, settings, pause, win and lose screens) connects the minigames into one game. Two of the minigames use artificial intelligence that runs directly on the phone, with no internet needed.

Who made what

Part	Main author	Sensors used
Egg Run	Lina	Gravity sensor / accelerometer (tilt), accelerometer (shake), front camera + light + proximity (sun shade), touchscreen, vibration
Sound Copy	Ons	Microphone, accelerometer (raise to record), gyroscope (spin the phone), touchscreen, vibration
Photo Challenge	Eya	Rear camera, touchscreen, vibration
Shared menus and integration	Lina	Touchscreen, Android back button

1. The big picture

How a player moves through the game

Main menu → Play → Level list → Pick a minigame → Play it → Win screen / Lose screen → Next game · Retry · Levels · Menu

During a minigame the player can open the **pause menu** (Resume, Restart, Settings, Levels, Main menu). **Settings** can also be opened from the main menu and the level list. Every minigame is unlocked from the start: the player can pick any of them.

The main building blocks (simple version)

Block	What it does, simply
Menu scenes	Each menu is its own Unity scene: `MainMenu`, `LevelSelect`, `SettingsMenu`, `PauseMenu`, `WinMenu`, `LoseMenu`. Pause and Settings open *on top of* the current game, so the game is not lost while paused.
GameFlow (the "brain")	One object that stays alive for the whole game. It loads scenes with a black fade, handles pause, the Android back button, the screen orientation, music, and shows the Win/Lose screens.
MiniGameBridge	The only thing a minigame needs to talk to the menus. A minigame just calls `MiniGameBridge.Win(...)` or `MiniGameBridge.Lose(...)`, and the menus do the rest.
MiniGame Registry	The list of minigames shown in the level menu: name, description, scene, music, orientation (all landscape), and whether to hide the shared pause button.
Settings (GameSettings)	Music volume, sound-effects volume, vibration on/off, invert tilt, tilt sensitivity, sun events on/off (Egg Run), show debug info, reset progress. Saved on the phone.
Saves (SaveData)	For each minigame: completed or not, best number of stars (0 to 3) and best score. Saved on the phone with Unity's `PlayerPrefs`.
Audio	Music for the menus and for each minigame, click sounds on every button, win/lose jingles and game sound effects. All sounds are original, generated by a script.

> **Why this design?**
> Three different people made three different games. The bridge + registry design means a minigame does not need to know anything about the menus: it reports "won" or "lost" and the shared system shows the right screen, saves the stars and plays the right music. Adding a fourth game is just: add its scene, add one entry to the registry, call `Win` / `Lose`.

How stars and scores are given

Minigame	3 stars	2 stars	1 star	Saved score

Egg Run	finish with ≥ 30 s left	≥ 15 s left	finish in time	seconds left × 10
Sound Copy	winner's average AI accuracy ≥ 70%	≥ 45%	below 45%	winner's average accuracy
Photo Challenge	winner found 3 objects	2 objects	0–1 object	winner's points

2. Phone sensors explained simply

A **sensor** is a tiny part inside the phone that measures something about the real world and gives the game a number. The game reads these numbers many times per second and turns them into actions.

Sensor	What it measures (simple words)	Used in
Accelerometer	How the phone is being pushed or moved, *including gravity*. When the phone is still, it only "feels" gravity pulling down, so it tells which way is down. When you shake it, the numbers jump.	Egg Run (tilt backup, shake), Sound Copy (raise to record, is the phone flat?)
Gravity sensor	Only the direction of gravity, already cleaned of shaking. It is a "virtual" sensor: Android computes it from the accelerometer and gyroscope. Best for tilting.	Egg Run (tilt the spoon)
Gyroscope	How fast the phone is *turning* (rotation speed), in every direction.	Sound Copy (spin the phone on the table)
Microphone	Sound. The game records the sound wave as numbers.	Sound Copy (record the imitations)
Camera (front / rear)	Images. A picture is a grid of coloured dots (pixels).	Egg Run (front camera: is it covered?), Photo Challenge (rear camera: photos)
Light sensor	How bright it is around the phone, in lux.	Egg Run (backup sun-shade detection)
Proximity sensor	Whether something is very close to the top of the phone ("near" / "far"), like your ear during a call.	Egg Run (backup sun-shade detection)
Touchscreen	Where and when fingers touch the screen.	All games (buttons, walk button)
Vibration motor	Not a sensor but an *actuator*: it makes the phone buzz. Used as feedback you can feel.	All games

> **Important discovery: "virtual" sensors on budget phones**
>
> On our test phone (Samsung Galaxy A17), Samsung lists the light and proximity sensors as **virtual**: there is no real sensor chip behind the glass, Android only estimates the value. When we covered the top of the phone, the light reading barely changed. That is why Egg Run's "cover the sun" mechanic uses the **front camera** as its main signal (a thumb on the lens turns the picture dark), and keeps the light and proximity sensors only as backups.

How Unity reads the sensors

- The project uses Unity's **new Input System** package (the old input system is turned off, because Android builds should use only one).

- Sensors are **off by default** to save battery. Each script turns on the sensor it needs (`InputSystem.EnableDevice`) and reads it every frame.
- The camera is read with `WebCamTexture` and the microphone with Unity's `Microphone` class.
- On Android, the **camera and microphone need permission**. The game asks the first time (a system pop-up). If the player refuses, the game shows a message or a fallback button instead of crashing.
- Raw sensor values are noisy. The scripts **smooth** them (keep a running average) and use **thresholds with a "hold time"** so one random spike does not trigger an action.

Common tricks used everywhere (worth knowing for questions)

Trick	Simple explanation	Where
Smoothing	Blend the new reading with the previous one so the value moves gently instead of jumping.	Tilt, gravity in Sound Copy, camera room brightness
Dead zone	Ignore tiny movements (under 1°) so a still hand does not move the spoon.	Egg Run tilt
Calibration	Take the way the player holds the phone at the start as "level".	Egg Run tilt
Hysteresis	Use two thresholds (one to switch on, a different one to switch off) so a value hovering near the limit does not flicker.	Light sensor, camera covered
Hold time / debounce	The condition must stay true for a short time (0.12–0.5 s) before it counts.	Cover detection, raise to record, spin stop
Fallbacks	If a sensor is missing (in the Unity Editor, or a phone without it), use a keyboard key or an on-screen button instead.	Everywhere

3. Egg Run by Lina

> **In one sentence**
>
> You carry an egg on a spoon and walk to the finish flag before 60 seconds run out. Tilt the phone to keep the egg balanced, shake the phone to scare away a bird, and cover the front camera to shade the egg from the sun.

How to play

- **Start**: press Start. At that moment the game records how you hold the phone and calls it "level" (calibration).
- **Walk**: hold the round walk button (bottom-right). The player walks at 1.6 m/s, speeding up and slowing down smoothly.
- **Balance**: tilt the phone left/right; the spoon tilts the same way and the egg rolls by real physics.
- **Win**: reach the finish line. **Lose**: the egg falls (touches anything that is not the spoon), the egg gets cooked by the sun, or the 60 s run out.

Mechanic 1: Tilt to balance gravity sensor, accelerometer backup

Simply: the phone knows which way is down. If you tilt it to the right, "down" moves to one side of the phone, and the game turns that into an angle for the spoon.

- Reads the **gravity sensor**; if the phone has none, the **accelerometer**; in the Editor, the **A/D or arrow keys**.
- Angle = `asin(gravity.x / |gravity|)`, i.e. the side-to-side tilt in degrees. Works whether the phone is upright or flatter.
- Calibrated at Start, dead zone 1°, limited to ±45°, smoothed (smoothing 12).
- Settings: **Invert tilt** (for phones that report the other way) and **Tilt sensitivity** (×0.5 to ×1.5).
- The spoon cannot turn faster than 360°/s, so a sensor spike cannot fling the egg.

Mechanic 2: Walking makes the spoon unsteady

Simply: like a real egg-and-spoon race, walking shakes your hand, so you must keep correcting with the tilt. If you never tilt, the egg falls.

Effect	Value	What it feels like
Step rocking	18° in rhythm with steps	left-right rock each step
Sway	10°, random	shaky hand
Drift	up to 12°, slow	a lean that lasts seconds: the main reason the egg rolls off if you don't react
Stumble	10–18° every 2.5–5 s of walking	a sudden lurch, with a small vibration and sound

Bounce	0.08 m up/down	steps lift the spoon

All of these are multiplied by **Walk Difficulty** (0 to 2) and only happen while walking: standing still is a way to recover.

Mechanic 3: The bird `accelerometer (shake)`

Simply: a bird flies onto the spoon rim, its weight tips the spoon, and it pecks the egg. Shake the phone to scare it away.

- Comes in when the player passes x = 12, 28 and 44 m. Lands in 1.2 s.
- Its weight tips the spoon 6°, plus 3° per second it stays, up to 20°. It pecks the egg every 1.5 s.
- **Shake detection**: the script separates slow movement (gravity) from fast movement (jolts). A "jerk" is a jolt stronger than 0.8 g; **3 jerks within 0.8 s** = a shake. After a shake, there is a 0.5 s cooldown.
- While shaking, the tilt is **frozen** (held at its last value for 0.35 s after the last jerk), so shaking to scare the bird does not throw the egg off.
- Editor: press **S** to fake a shake.

Mechanic 4: Sun and heat `front camera (main), light + proximity sensors (backup)`

Simply: every so often the sun comes out. The screen becomes almost white (you can't see the egg) and the egg starts to cook. Put your thumb over the front camera to make shade: the glare fades so you can see again, and the egg cools down. If the heat bar fills up, the egg is cooked and you lose.

Step	Timing	What happens
First sun	8–12 s after the start	then every 9–15 s; never in the last 6 s
Warning	1.5 s	a sun rises in the corner, the screen brightens, the phone buzzes twice, a chime plays
Sun	5–8 s	glare covers 96% of the screen; covered = only 10% glare plus a dark "shadow"
Heat	4.5 s of full sun to cook	cools fully in 5 s of shade; a heat bar under the timer shows it, the egg turns brown, a sizzle sound plays

How "covered" is detected (camera)

- The front camera runs at a tiny size (160×120). Ten times per second, the script measures the picture's **average brightness** and **contrast** (how different the pixels are). Nothing is saved or shown.
- It learns the room's normal brightness slowly (6 s to follow darker, 0.5 s to follow brighter), and never while covered.
- **Covered** if: brightness is under 40% of the room's brightness, *or* the picture is almost black (under 0.06), *or* the picture is flat and dim (contrast under 0.035 and dimmer than the room). The last rule matters because a phone camera brightens itself automatically after a few seconds behind a thumb.
- Uncovered again only above 55% (hysteresis). A cover must last 0.12 s to count.

Backups and fallbacks

- **Light sensor**: covered when light drops under 25% of the room's level (uncovered above 40%). Not used in a dark room (under 12 lux).
- **Proximity sensor**: covered when it reports "near".
- While the camera works, the light and proximity sensors are ignored (they can be virtual and unreliable).
- No camera permission and no usable sensor: an on-screen **SHADE** button appears (hold it). Editor: hold **L**.
- The sun can be switched off in Settings (**Sun events**).

Vibration and sound in Egg Run

Short buzzes: egg breaks (long buzz), bird lands, bird pecks, sun warning, stumbles, heat danger. Sounds: egg crack, stumble, bird flaps and chirps, sun warning chime, sizzle, plus the Egg Run music loop.

Debug helpers

- **Show debug info** (Settings) shows the tilt angle and its source, shake strength, camera brightness and room brightness, light and proximity values, the sun phase and the heat.

- It also opens a **sensor panel** listing every sensor the phone exposes, with live values. Useful to prove which sensors exist on a phone.

> **Why these sensors make sense here**
>
> Tilt = balancing a real spoon. Shake = shooing a real bird. Covering the camera = shading your eyes and the egg from the sun with your hand. Each sensor matches a natural real-world gesture, so the player understands it without a tutorial.

4. Sound Copy by Ons

> **In one sentence**
>
> A 2-player party game on one phone: watch a short meme video, both players imitate its sound into the microphone, and an AI on the phone decides whose imitation is closer to the original.

How a match goes (always 3 rounds)

Home → Setup: names + category → Round screen → Spin the phone → Watch the video → Pass phone → record (1st) → Pass phone → record (2nd) → AI analysis → Verdict → … 3 rounds … Results → Pocket Jungle Win screen

- **Categories**: International (4 videos), Tunisian (3 videos), Mixed (random from both). A video is never repeated twice in a row.
- **Points**: 1 point for winning a round, 2 on a "x2 round". A tie (AI scores less than 3 points apart) gives no point.
- **Match winner**: most points; if tied, the higher average AI accuracy; if still within 1%, it is a draw.

Mechanic 1: Spin the phone gyroscope (+ accelerometer to check the phone is flat)

Simply: like "spin the bottle". Put the phone flat on the table between the two players (Player 1 on the bottom side of the screen, Player 2 on the top side) and spin it. When it stops, the player the top of the screen points to goes first. A fast spin makes the round a x2 round.

- The gyroscope gives the rotation speed around the axis that goes through the screen. The script adds it up over time to know **how far the phone turned**.
- "Spinning" starts above 0.6 turns per second. "Stopped" = below 0.15 turns/s for 0.5 s.
- A spin must reach at least 0.8 turns/s (otherwise "Spin harder!"). Reaching **2.2 turns/s** = **x2 round**.
- Who goes first: if the phone ended up turned less than a quarter turn either way from where it started, its top still faces Player 2 (cosine of the angle ≥ 0), so Player 2 starts; otherwise Player 1.
- The accelerometer checks the phone is lying flat (screen facing up) before a spin counts.
- Fallback: "Spin for me" button (random result). Editor: **Space**.
- **Why it matters**: going second is an advantage (you hear the first imitation), so the order must be fair and random. It replaced an earlier "shake for a joker" idea, which did not match the theme of the game.

Mechanic 2: Raise to record accelerometer

Simply: you hold the phone like a microphone. Raising it to your mouth starts the 3-2-1 countdown by itself; lowering it during the countdown cancels.

- "Upright" is measured from gravity: 0 = phone lying flat, 1 = phone standing straight up (`1 - |gravity.z|`, using smoothed gravity).
- **Raised** = upright ≥ 0.6 held for 0.3 s. The phone must have been **down** (≤ 0.4) at least once first, so it never starts by itself.

- Lowered during the 3-2-1 for 0.4 s = take cancelled. The countdown also buzzes (you cannot see the screen at your mouth).
- The microphone button still works as a backup. Editor: hold **R**.

Mechanic 3: The recording `microphone`

- Records at 44,100 samples per second, for **exactly the length of the video**.
- A **live level meter** moves with your voice (it measures the loudness of the last ~23 ms of sound), and a progress bar shows the time left.
- After recording: **Play back** or **Redo** (once per player per round), then Next player / Let the AI judge.
- The game **music pauses** during the video, hand-over, recording and analysis, so it is never recorded by the microphone.

The AI judge (how the winner is decided)

> **Simply**
> The AI turns each sound (the original and the two imitations) into a list of numbers that describes it, a "fingerprint". It then measures how close each imitation's fingerprint is to the original's. The closer, the higher the score (0–100%). Everything happens **on the phone**, offline.

Part	Weight	What it compares
Acoustic detail	60%	the "colour" of the sound: which pitches are loud (from a spectrogram)
Temporal shape	25%	how the sound changes over time (rhythm, when it gets loud or quiet)
YAMNet	15%	a neural network made by Google that recognises 521 kinds of sounds (speech, laughter, animals, horns…). Two sounds that YAMNet "hears" as the same kind score higher.

- **Steps**: convert to mono at 16 kHz → compute a **log-Mel spectrogram** (a picture of which frequencies are loud over time, 64 frequency bands) → YAMNet looks at 96-frame patches (~1 s each, up to 12 patches) → compare with **cosine similarity**.
- The final score is multiplied by how active the recording is. A recording that is almost silent (activity under 0.12) scores 0 and is marked "Too quiet".
- YAMNet runs with Unity's AI engine (**Sentis / Inference Engine**) from an ONNX model file (`YamNet.onnx`). Each video has a matching reference sound file (WAV, mono 16 kHz) extracted with FFmpeg.
- **Limit to know**: YAMNet is a general sound classifier, not built to judge imitations; that is why it only counts for 15% and the acoustic and temporal comparisons carry most of the decision. The weights were tuned by testing.

Other features

- **Hand-over screen**: full screen in the player's colour, "Pass the phone to [name]", "I'm ready" button.
- **Verdict screen**: both players' scores fill up as animated bars, the winner's card pops, "+1" or "+2 x2 ROUND" or "TOO QUIET" tags.
- **Results screen**: trophy in the winner's colour, final score, a round-by-round table and the average accuracy; Rematch or Continue (opens Pocket Jungle's Win screen).
- **Pause** (its own): Resume, Restart, Quit match, Games menu. The shared pause button is hidden in this game.
- Player names are remembered for next time.
- The interface is built by an editor tool (`SoundCopy > Build Scenes`), then restyled by hand with the same art pack and fonts as the Pocket Jungle menus. A second tool (`Add Spin Screen`) adds new screens without overwriting hand edits.

Why these sensors make sense here

Microphone = the game is about your voice. Raising the phone = holding a microphone, which frees the screen taps and feels like a performance. Spinning = a classic party ritual that makes the turn order fair and visible to everyone.

5. Photo Challenge by Eya

> **In one sentence**
>
> Two players take turns: each turn the game names an everyday object, and the player has 15 seconds to find it and photograph it; an AI checks whether the photo really shows that object.

How to play

- Enter both names, then **6 turns** in total (3 per player, alternating).
- Each turn starts with "Pass the phone to [name]". The object's **name and reference picture are hidden** ("???") until the player taps **Je suis prêt** (I'm ready). They appear at the same moment the **15-second timer** starts, which is fair: nobody can start searching early.
- Point the camera at the object and tap **Prendre la photo** (take the photo). If the AI says it matches: **+1 point**, next player. If not: try again until time runs out.
- Objects: a bottle, a chair, a cup, a book, a phone. They are drawn like shuffled cards, so each appears before any repeats.
- End: the player with the most points wins (or a draw). Continue opens Pocket Jungle's Win screen.

Sensor: the rear camera

- Uses the **rear camera** (back of the phone), requested at 1280×720, 30 images per second, shown live in the middle of the screen.
- Android asks for camera permission the first time.
- **Rotation fix**: phone cameras send their picture rotated or mirrored depending on the phone. The game reads the camera's reported rotation (`videoRotationAngle`) and mirroring and turns both the live preview and the captured photo the right way up. Playing in landscape also helps, because phone camera sensors are naturally landscape.

The AI check (MobileCLIP)

> **Simply**
>
> An AI model turns the photo into a list of numbers that describes what is in the picture (its "meaning"). It does the same for a reference picture of the object. If the two lists point in a similar direction, the pictures show similar things.

- Model: **MobileCLIP-S2 image encoder** (`vision_model_fp16.onnx`, ~72 MB), run on the phone with Unity Sentis (on the graphics chip, or the processor if that fails).
- The photo is cropped to a square from the centre and resized to 256×256 pixels before going into the model.
- **Cosine similarity** between the photo's numbers and the reference's numbers: 1 = identical meaning, 0 = unrelated. A match needs **≥ 0.45**. Every comparison is written to the Console so the threshold can be tuned.

- Reference images are prepared once when the game starts (the model loads in the background; "Start" waits until it is ready).

Feedback

Sounds: a whoosh when the object is revealed, a camera shutter, a ding on a match, a buzz on a miss or time-out, ticks for the last 3 seconds, the Photo Challenge music loop. Vibration on a match.

> **Known limits (good to say honestly)**
>
> The AI compares the photo with *one specific* reference picture, so a very different-looking bottle, or a messy background, can fail. The 0.45 threshold should be tuned with real photos. The project also contains an unused object-detection model (YOLO, `yolo11n.onnx`) that could recognise 80 kinds of objects; switching to it is the main planned improvement. Photo Challenge's texts are in French, while the other games are in English.

> **Why this sensor makes sense here**
>
> The camera turns the real room into the game board: players must physically move around and find real objects.

6. Shared systems

Menus and navigation

- The game starts on **MainMenu** (logo, Play, Settings, Quit). Play opens the **LevelSelect** list of minigame cards with stars earned.
- Scene changes use a short **black fade**.
- **Pause**: stops game time (`Time.timeScale = 0`) and mutes game audio; the pause menu loads on top of the game.
- **Android back button**: closes Settings, then the pause menu, then pauses the game; in menus it goes back to the main menu; on the main menu it quits.
- **Orientation**: the whole game is landscape. Each minigame's orientation is a setting in the registry, so a future portrait game is possible.
- **Safe area**: the interface stays clear of the camera notch and rounded corners.

Settings (saved on the phone)

Setting	Default	Effect
Music	80%	music volume
Sound effects	100%	clicks, jingles, game sounds
Vibration	on	all buzzes in every game
Invert tilt	off	flips Egg Run's tilt direction
Tilt sensitivity	×1.0	×0.5 (gentle) to ×1.5 (sharp)
Sun events (Egg Run)	on	turns the sun and heat mechanic on/off
Show debug info	off	sensor readouts and the sensor panel
Reset progress	-	tap twice to erase stars and best scores

Sound and music

- 4 music loops (menus, Egg Run, Sound Copy, Photo Challenge) and 17 sound effects, all **original**, generated by a Python script (`Tools/AudioSynth/synth.py`) from simple waveforms; no downloaded or copyrighted audio.
- Sounds are loaded **by file name** from `Assets/UI/Resources/Audio` ; replacing a file with another of the same name changes the sound.
- Every button gets a click sound automatically, including buttons built by code.

Visual style

- A green cartoon UI pack ("2D Casual UI"), jungle backgrounds, and the fonts **Starborn** (titles) and **Cute Dino** (text), used in the menus and in all three minigames.
- Logo "Pocket Jungle": a wooden sign with leaves, a sun and badges for the three games.

- Layout: 1920×1080 reference, main buttons at the bottom-right under the right thumb, large text and buttons for a phone screen.

Permissions the app asks for

Permission	Why	If refused
Camera	Egg Run shade detection, Photo Challenge photos	Egg Run shows a SHADE button; Photo Challenge shows a message
Microphone	Sound Copy recordings	message "Mic access denied", can retry from Settings

Motion sensors (accelerometer, gravity, gyroscope, light, proximity) need no permission on Android.

7. Testing tools and building the app

Keyboard shortcuts in the Unity Editor (no phone needed)

Key	Game	Fakes
A / D or arrows	Egg Run	tilting the phone
S	Egg Run	a shake (scare the bird)
L (hold)	Egg Run	covering the camera / sensor
R (hold)	Sound Copy	raising the phone to your mouth
Space	Sound Copy	a spin of the phone
Esc	All	the Android back button

On the phone

- Turn on **Show debug info** to see live sensor values and the list of sensors the phone really has.
- Unity Remote (phone as a controller for the Editor) does *not* pass the light sensor or a working camera, so those features must be tested in a real build.
- Thresholds (cover ratio, raise level, spin speeds, AI threshold) are all adjustable in the Unity Inspector without changing code.

Building for Android

- Unity 6000.6.3f1 with Android Build Support; scripting backend IL2CPP (C# converted to C++ then compiled for the phone).
- Input: "Input System Package (New)" only. The old input system was turned off because Android warns against using both.
- All scenes are added to the build list automatically (`Tools > Game UI > Sync Build Scene List`).
- Problem we met: the antivirus (RAV) blocked Unity's build tools and the Unity installer. Fix: allow Unity's tools in the antivirus rules (or pause it during the build).

Teamwork and version control

- The project uses Git and GitHub. Each teammate worked on a branch; their branches were merged into one project while keeping everyone's files (an earlier merge would have deleted Egg Run, so it was merged carefully).
- Each colleague's folder contains a change log (`CHANGEMENTS_INTEGRATION.md`) explaining what was changed during integration.

8. Validation questions and answers

Short, simple answers you can give out loud. Grouped by topic.

General

What is your project?

Pocket Jungle, a mobile game made of three minigames that use the phone's sensors as controls: Egg Run (tilt, shake, camera), Sound Copy (microphone, accelerometer, gyroscope, on-device AI) and Photo Challenge (camera, on-device AI), linked by shared menus.

Why use sensors instead of buttons?

Each sensor matches a natural real-world gesture (balancing a spoon, shooing a bird, shading from the sun, holding a microphone, spinning a bottle, taking a photo). It is more fun and more physical, and it shows what a phone can do beyond a touchscreen.

How do the three games become one game?

A shared menu system. Each minigame only calls Win or Lose through a "bridge"; the shared system shows the result screen, saves stars and best scores, plays music, handles pause and moves to the next game. A registry lists the games for the level menu.

Does it need the internet?

No. Everything, including both AI models, runs on the phone.

Which phone did you test on, and what did you learn?

A Samsung Galaxy A17. Its light and proximity sensors are "virtual" (estimated by Android, no real chip), so covering them barely changed the values. We switched Egg Run's shade detection to the front camera, which works on every phone.

Sensors (general)

What is the difference between the accelerometer, the gravity sensor and the gyroscope?

The accelerometer measures all pushes on the phone including gravity (so it is shaky). The gravity sensor gives only the gravity direction, cleaned by Android using several sensors (good for tilt). The gyroscope measures how fast the phone rotates (good for spinning).

How do you avoid false triggers from noisy sensors?

Smoothing (running average), dead zones (ignore tiny moves), two thresholds for on/off (hysteresis), and requiring the condition to last a short time (0.12–0.5 s) before it counts.

What happens on a phone without a sensor?

Every sensor feature has a fallback: Egg Run tilt uses the accelerometer if there is no gravity sensor; the sun uses light/proximity if no camera, or a SHADE button; Sound Copy has a mic button and a "Spin for me" button. In the Editor, keyboard keys replace sensors.

How do you read sensors in Unity?

With the new Input System package: enable the device (sensors are off by default to save battery) and read its value each frame. Camera with WebCamTexture, microphone with the Microphone class.

Do sensors drain the battery?

Sensors are turned on only by the scripts that need them. The camera in Egg Run runs at a tiny 160×120 size and stops at the end of the round.

Egg Run

How is the tilt angle calculated?

From the gravity direction: angle = arcsin(gravity x / gravity length), in degrees. It is calibrated when you press Start (your current grip counts as level), with a 1° dead zone, a ±45° limit and smoothing.

How does the egg move?

With Unity's physics engine: the egg is a real physics object that rolls on the spoon under gravity. It breaks as soon as it touches anything that is not the spoon.

Why does the egg fall if I don't tilt?

Walking adds rocking, sway, a slow drift and random stumbles to the spoon, like a real hand. The drift is the key: it leans the spoon for several seconds, so you must correct it.

How is a shake detected?

The script separates slow changes (gravity) from fast jolts. A jolt stronger than 0.8 g is a "jerk"; three jerks within 0.8 seconds make a shake. During a shake the tilt is frozen so the egg stays safe.

How do you know the camera is covered?

The front camera's picture is measured ten times per second: if its brightness falls under 40% of the room's normal brightness, or it is almost black, or it is flat and dim (a thumb gives a blurry, even picture), it counts as covered. The room's brightness is learned slowly while uncovered.

Do you record or store the camera image?

No. Only the average brightness and contrast numbers are computed; the picture is never shown or saved.

How are stars given?

Finish within 60 seconds: 3 stars with 30 s or more left, 2 stars with 15 s or more, otherwise 1 star. Score = seconds left × 10.

Sound Copy

How does the AI decide the winner?

It turns the original sound and both imitations into "fingerprints" and measures how close each imitation is to the original: 60% sound detail (spectrum), 25% shape over time, 15% YAMNet (a Google model that recognises 521 sound types). Silent recordings get 0; a difference under 3 points is a tie.

What is a spectrogram / log-Mel?

A picture of a sound: time from left to right, pitch from bottom to top, brightness = loudness. "Mel" spaces the pitches like human hearing; "log" compresses loudness like our ears do.

What is cosine similarity?

A way to compare two lists of numbers by the angle between them: 1 means they point the same way (very similar), 0 means unrelated. Used in both AI games.

Why only 15% for YAMNet?

YAMNet classifies sound types; it was not trained to judge imitations. It helps (e.g. "this is a horn") but the direct acoustic and timing comparisons decide most of the score.

How does "raise to record" work?

The accelerometer gives gravity; when the phone stands up like a microphone (uprightness ≥ 0.6 for 0.3 s) after having been down, the countdown starts. Lowering it during the countdown cancels.

How does the spin decide who starts?

The gyroscope gives the rotation speed; adding it up over time gives how far the phone turned. If the top of the screen still points to Player 2's side (turned less than a quarter turn either way), Player 2 starts; otherwise Player 1. Top speed ≥ 2.2 turns per second = x2 round.

Why not keep the music on?

The microphone would record it and the AI would compare music instead of the voice, so the music pauses during the video, recording and analysis.

Why is the match fixed to 3 rounds?

It keeps matches short and equal, and with the x2 rounds there are still comebacks.

Photo Challenge

How does the game know the photo shows the right object?

MobileCLIP turns the photo and the reference picture into lists of numbers describing their content; if their cosine similarity is at least 0.45, it is a match.

Why is the object hidden until Ready?

Fairness: otherwise a player could read the object and start searching before their 15 seconds begin, while the other player is still holding the phone.

Why was the camera image sideways before?

Phone cameras send images rotated or mirrored depending on the phone. We now read the reported rotation and mirroring and correct both the preview and the photo sent to the AI.

What would you improve?

Use the YOLO object detector (already in the project) to recognise "any cup" instead of comparing with one reference picture, add more objects, and tune the threshold with real photos.

Technical / Unity

Which Unity version and render pipeline?

Unity 6 (6000.6.3f1) with the Universal Render Pipeline (URP), built for Android with IL2CPP.

How do you run AI models in Unity?

With Unity Sentis (Inference Engine). Models are ONNX files (YAMNet for sounds, MobileCLIP for images); they run on the phone's graphics chip, or the processor as a backup.

How is pause implemented?

Time.timeScale = 0 stops everything that uses game time (physics, timers), game audio is paused, and the pause menu scene is loaded on top so the game keeps its state.

How is progress saved?

With PlayerPrefs (small key/value storage on the phone): per game, completed, best stars and best score; plus all settings.

How did you organise the code?

One folder per minigame plus a shared UI folder (Core, Menus, Editor tools). Minigames talk to the menus only through MiniGameBridge. Settings are in one static class. Editor tools build the menus and scene lists automatically.

How do you handle different screen sizes?

A Canvas Scaler with a 1920×1080 reference (screens keep their size on any phone), a safe area for notches, and windows that shrink to fit on wide phones.

Where do the sounds and music come from?

We generated them with our own Python script from simple waveforms (sine, square, noise), so there are no copyright issues. They can be swapped by replacing files with the same name.

Problems we faced

What were the biggest difficulties?

(1) Virtual light/proximity sensors on the test phone: solved with the camera. (2) Merging three separate projects without losing anyone's work. (3) Camera rotation on Android. (4) The antivirus blocking Unity's build and install. (5) Keeping the walking difficulty fair: the egg first never fell, so we added drift and stumbles.

What are the current limitations?

Thresholds were tuned on one phone; the AI judges are approximations (sound imitation and photo matching); Photo Challenge's texts are still in French; the fourth minigame slot is empty.

What would you add next?

A fourth minigame, YOLO detection in Photo Challenge, a "blind round" in Sound Copy that plays the sound's rhythm with vibration, tuning on more phones, and professional sounds.

9. Glossary

Word	Meaning in one sentence
Accelerometer	Sensor that measures pushes on the phone, including gravity.
Actuator	A part that acts on the world instead of measuring it (the vibration motor).
Calibration	Recording a starting position as "zero" (Egg Run: your grip counts as level).
Canvas Scaler	Unity component that resizes the interface to fit any screen.
Cosine similarity	A 0–1 score of how alike two lists of numbers are, from the angle between them.
Dead zone	A small range of values that is ignored so tiny movements do nothing.
Debounce / hold time	A condition must last a short time before it counts.
Embedding	The list of numbers an AI model produces to describe an image or sound.
g	The strength of Earth's gravity (1 g); used to measure shake strength.
Gravity sensor	Virtual sensor giving only the direction of gravity, without shaking.
Gyroscope	Sensor that measures how fast the phone rotates.
Hysteresis	Different thresholds to switch on and off, to avoid flickering.
IL2CPP	Unity's way of converting C# code to C++ to build fast Android apps.
Inference	Running a trained AI model on new data (here, on the phone).
Input System	Unity's modern package for reading touch, keyboard and sensors.
Log-Mel spectrogram	A picture of a sound: time × pitch, with loudness on a human-like scale.
Lux	Unit of light brightness.
MobileCLIP	A small AI model (Apple) that describes images as numbers; used in Photo Challenge.
ONNX	A standard file format for AI models, readable by Unity Sentis.
Physics engine	The part of Unity that simulates gravity, rolling and collisions (the egg).
PlayerPrefs	Unity's small storage on the phone for settings and saves.
Proximity sensor	Sensor that says if something is very close to the top of the phone.
Registry	The list of minigames shown in the level menu.
Scene	A Unity "level" or screen (a menu or a minigame).
Sentis / Inference Engine	Unity's package for running AI models inside a game.
Smoothing	Blending each new value with the previous ones so it moves gently.
Threshold	The value a reading must cross for something to happen.

Time.timeScale	Unity's game speed; 0 = paused.
URP	Universal Render Pipeline: Unity's graphics system for mobile-friendly rendering.
Virtual sensor	A value Android computes from other sensors instead of a real chip.
WebCamTexture	Unity's way to get live images from a phone camera.
YAMNet	A Google AI model that recognises 521 types of sounds; used in Sound Copy.